

Abstract Factory Design Pattern in C# with Examples

In this article, I will discuss the **Abstract Factory Design Pattern in C#** with Examples. Please read our previous article discussing the [Factory Method Design Pattern in C#](#) with an example. The Abstract Factory Design Pattern belongs to the creational design pattern category and is one of the most used design patterns in real-world applications. As part of this article, we will discuss the following things.

1. [What is an Abstract Factory Design Pattern?](#)
2. [Example to understand the Abstract Factory Pattern.](#)
3. [Implementing Abstract Factory Design Pattern in C#.](#)
4. [Differences Between Abstract Factory and Factory Method Design Pattern.](#)

What is the Abstract Factory Design Pattern?

According to the Gang of Four Definition, **The Abstract Factory Design Pattern provides a way to encapsulate a group of factories with a common theme without specifying their concrete classes.**

Abstract means hiding some information, factory means which produces the products, and pattern means a design. So, the Abstract Factory Pattern is a software design pattern that provides a way to encapsulate a group of individual factories that have a common theme.

In simple words, the Abstract Factory is a super factory that creates other factories. It is also called the Factory of Factories. The Abstract Factory design pattern provides an interface for creating families of related or dependent products but leaves the actual object creation to the concrete factory classes.

Components of Abstract Factory Design Pattern:

The Abstract Factory Design Pattern consists of the following components:

- **AbstractFactory:** Declares an interface for operations that create abstract products. This will be an interface for operations that will create Abstract Product objects.
- **ConcreteFactory:** Implements the operations to create concrete product objects. These classes implement the Abstract Factory interface and provide implementations for the interface methods. We can use these concrete classes to create concrete product objects.
- **AbstractProduct:** Declares an interface for a type of product object. These are going to be interfaces for creating abstract products. Here, we need to define the Operations a Product should have.
- **ConcreteProduct:** Implements the AbstractProduct interface. These are the classes that implement the Abstract Product interface.
- **Client:** Uses interfaces declared by AbstractFactory and AbstractProduct classes. This class will use our Abstract Factory and Abstract Product interfaces to create a family of products.

Example to Understand Abstract Factory Design Pattern using C#:

Let us understand the Abstract Factory Design Pattern with one Real-Time Example. First, we will implement the example and then compare the Example with the Abstract Factory Design Pattern UML diagram so you can easily understand the concept.

We want to implement one application for showing the Car and Bike details. Here, we want to show two types of bikes, i.e., Regular Bikes and Sports Bikes, and the details of two types of cars, i.e., Regular and Sports cars. You must remember that a sports bike and car belong to the same family called Sports. Similarly, Regular Bikes and Regular Cars belong to the same family called Regular. With this kept in mind, let us proceed and see how we can implement this using the Abstract Factory Design Pattern in C#.

Step 1: Creating Abstract Products

Here, we need to declare interfaces for creating abstract products. As we will create two types of familiar products, such as Bikes and Cars, here we need to create two interfaces or abstract classes representing each abstract

product type. Here, I am going to create two interfaces.

IBike.cs (AbstractProductA)

Create an interface named **IBike.cs** and copy and paste the following code. This is going to be one of the Abstract Products. Each distinct product of the Bike product family should have a base interface. In this case, all variants of Bike products (SportsBike and RegularBike) must implement this IBike interface. As you can see, we have created the following interface with one abstract method. That method will be implemented by the subclasses (SportsBike and RegularBike) of the IBike interface.

```
namespace AbstractFactoryDesignPattern
{
    // The AbstractProductA interface
    // Each distinct product of the Bike product family should have a base
    interface.
    // All variants of Bike products must implement this IBike interface.
    public interface IBike
    {
        void GetDetails();
    }
}
```

ICar.cs (AbstractProductB)

Create an interface with **ICar.cs** and then copy and paste the following code. This is going to be our second Abstract Product. Each distinct product of the Car product family should have a base interface. In this case, all variants of the Car products (SportsCar and RegularCar) must implement this ICar interface. As you can see, we have created the following interface with one abstract method, which will be implemented by the subclasses of the ICar interface.

```
namespace AbstractFactoryDesignPattern
{
    // The 'AbstractProductB' interface
    // Each distinct product of the Car product family should have a base
    interface.
    // All variants of the Car products must implement this ICar interface.
    public interface ICar
    {
        void GetDetails();
    }
}
```

Step 2: Creating Concrete Products

Here, we must define the concrete product object the corresponding concrete factory will create. How the concrete factory will create the product object will be discussed later in the article when we discuss the concrete factory component. Remember that the concrete product classes must implement the Abstract Product Interface. In our example, the concrete product class must implement the IBike or ICar interface. So, as per our requirements, let us create four concrete products, i.e., RegularCar, SportsCar, RegularBike, and SportsBike.

RegularBike.cs (ProductA1)

Create a class file named **RegularBike.cs**, then copy and paste the following code. The following RegularBike Product Belongs to the Bike product family. As you can see, it implements the IBike interface and provides the implementation for the GetDetails method.

```

using System;
namespace AbstractFactoryDesignPattern
{
    // The ProductA1 class
    // Concrete Products are going to be created by corresponding Concrete
    // Factories.
    // The following RegularBike Product Belongs to the Bike product family
    public class RegularBike : IBike
    {
        public void GetDetails()
        {
            Console.WriteLine("Fetching RegularBike Details..");
        }
    }
}

```

SportsBike.cs (ProductB1)

Create a class file named **SportsBike.cs**, then copy and paste the following code. The following SportsBike Product Belongs to the Bike product family. As you can see, it implements the IBike interface and provides implementations for the GetDetails method.

```

using System;
namespace AbstractFactoryDesignPattern
{
    // The ProductB1 class
    // Concrete Products are going to be created by corresponding Concrete
    // Factories.
    // The following SportsBike Product Belongs to the Bike product family
    public class SportsBike : IBike
    {
        public void GetDetails()
        {
            Console.WriteLine("Fetching SportsBike Details..");
        }
    }
}

```

RegularCar.cs (ProductA2)

Create a class file named **RegularCar.cs** and copy and paste the following code. The following RegularCar Product Belongs to the Car product family. As you can see, it implements the ICar interface and provides implementations for the GetDetails method.

```

using System;
namespace AbstractFactoryDesignPattern
{
    // The ProductA2 class
    // Concrete Products are going to be created by corresponding Concrete
    // Factories.
    // The following RegularCar Product Belongs to the Car product family
    public class RegularCar : ICar
    {
        public void GetDetails()
        {

```

```

        Console.WriteLine("Fetching RegularCar Details..");
    }
}

```

SportsCar.cs (ProductB2)

Create a class file named **SportsCar.cs** and copy and paste the following code. The following SportsCar Product Belongs to the Car product family. As you can see, it implements the ICar interface and provides implementations for the GetDetails method.

```

using System;
namespace AbstractFactoryDesignPattern
{
    // The ProductB2 class
    // Concrete Products are going to be created by corresponding Concrete
    // Factories.
    // The following SportsCar Product Belongs to the Car product family
    public class SportsCar : ICar
    {
        public void GetDetails()
        {
            Console.WriteLine("Fetching SportsCar Details..");
        }
    }
}

```

Step 3: Creating Abstract Factory

Here, we need to create an interface for operations that will create AbstractProduct objects. In our example, it is going to be IVehicleFactory. So, create a class file with the name **IVehicleFactory.cs** and then copy and paste the following. As you can see, this class contains two methods. The CreateBike method will create different variants of Bikes, whereas the CreateCar method will create different variants of Cars. The point you need to remember is that we are only declaring the methods and returning the type of these two methods to be abstract products, i.e., ICar and IBike. Hence, from the child classes, we can create different objects (objects of subclasses of ICar and IBike).

```

namespace AbstractFactoryDesignPattern
{
    // The AbstractFactory interface
    // The Abstract Factory interface declares a set of methods that return
    // different abstract products.
    // These products are called a family.
    // A family of products may have several variants
    public interface IVehicleFactory
    {
        //Abstract Product A
        IBike CreateBike();
        //Abstract Product B
        ICar CreateCar();
    }
}

```

Note: The above abstract factory class is used to create a family of other factories. Let's proceed to create such a family of factories.

Step 4: Creating Concrete Factories

Here, we must create concrete classes that implement the operations to create concrete product objects. These classes will implement the AbstractFactory interface and provide implementations for the two interface methods. In our example, we will create two Concrete Classes, i.e., RegularVehicleFactory and SportsVehicleFactory.

RegularVehicleFactory.cs (ConcreteFactory1)

The RegularVehicleFactory concrete class is going to create Regular Vehicle Concrete Products. In our example, the Regular Vehicle Concrete Products are RegularBike and RegularCar. So, create a class file named **RegularVehicleFactory.cs** and copy and paste the following code. If you notice, the signatures of the Concrete Factory's methods return an abstract product (IBike) and (ICar), while inside the method, a concrete product (new RegularBike and new RegularCar) is instantiated. So, the following Concrete Factory Produces Regular Bike and Regular Car Products belong to the same Regular Family.

```
namespace AbstractFactoryDesignPattern
{
    // The ConcreteFactory1 class
    // Concrete Factories produce a family of products that belong to a
    single variant.
    // The following Concrete Factory Produces Regular Bike and Car which
    are compatible
    // The signatures of the Concrete Factory's methods return an abstract
    product (IBike) and (ICar)
    // while inside the method a concrete product (new RegularBike and new
    RegularCar) is instantiated.
    public class RegularVehicleFactory : IVehicleFactory
    {
        public IBike CreateBike()
        {
            return new RegularBike();
        }
        public ICar CreateCar()
        {
            return new RegularCar();
        }
    }
}
```

SportsVehicleFactory (ConcreteFactory2)

The SportsVehicleFactory concrete class is going to create Sports Vehicle Concrete Products. In our example, the Sports Vehicle Concrete Products are SportsBike and SportsCar. So, create a class file named **SportsVehicleFactory.cs** and copy and paste the following code. If you notice, the signatures of the Concrete Factory's methods return an abstract product (IBike) and (ICar), while inside the method, a concrete product (new SportsBike and new SportsCar) is instantiated. So, the following Concrete Factory Produces Sports Bike and Sports Car Products that belong to the same Sports Family.

```
namespace AbstractFactoryDesignPattern
{
    // The ConcreteFactory2 class
    // Concrete Factories produce a family of products that belong to a
    single variant.
    // The following Concrete Factory Produces Sports Bike and Sports Car
    which are compatible
```

```

    // The signatures of the Concrete Factory's methods return an abstract
    product (IBike) and (ICar)
    // while inside the method a concrete product (new SportsBike and new
    SportsCar) is instantiated.
    public class SportsVehicleFactory : IVehicleFactory
    {
        public IBike CreateBike()
        {
            return new SportsBike();
        }
        public ICar CreateCar()
        {
            return new SportsCar();
        }
    }
}

```

Note: In our example, IVehicleFactory provides an interface to the client for creating families of related or dependent objects. Here, we have two concrete implementations of that interface: RegularVehicleFactory and SportsVehicleFactory classes. These two classes manufacture two different types of families of related objects, i.e., Bikes and Cars. The RegularCar and RegularBike classes belong to the RegularVehicleFactory family. On the other hand, the SportsCar and SportsBike classes belong to a SportsVehicleFactory family.

Step 5: Client

The Client class uses AbstractFactory and AbstractProduct interfaces to create a family of related objects. In our example, it will be the Main method of the Program class. So, modify the Main method of the Program class as follows.

```

using System;
namespace AbstractFactoryDesignPattern
{
    //Client Code
    class Program
    {
        public static void Main()
        {
            // Fetch the Regular Bike and Car Details
            // Creating RegularVehicleFactory instance
            IVehicleFactory regularVehicleFactory = new
RegularVehicleFactory();
            //regularVehicleFactory.CreateBike() will create and return
Regular Bike
            IBike regularBike = regularVehicleFactory.CreateBike();
            regularBike.GetDetails();
            //regularVehicleFactory.CreateCar() will create and return
Regular Car
            ICar regularCar = regularVehicleFactory.CreateCar();
            regularCar.GetDetails();
            // Fetch the Sports Bike and Car Details Created
            // Creating SportsVehicleFactory instance
            IVehicleFactory sportsVehicleFactory = new
SportsVehicleFactory();
            //sportsVehicleFactory.CreateBike() will create and return
Sports Bike

```

```

        IBike sportsBike = sportsVehicleFactory.CreateBike();
        sportsBike.GetDetails();
        //sportsVehicleFactory.CreateCar() will create and return
Sports Car
        ICar sportsCar = sportsVehicleFactory.CreateCar();
        sportsCar.GetDetails();
        Console.ReadKey();
    }
}
}

```

Output:

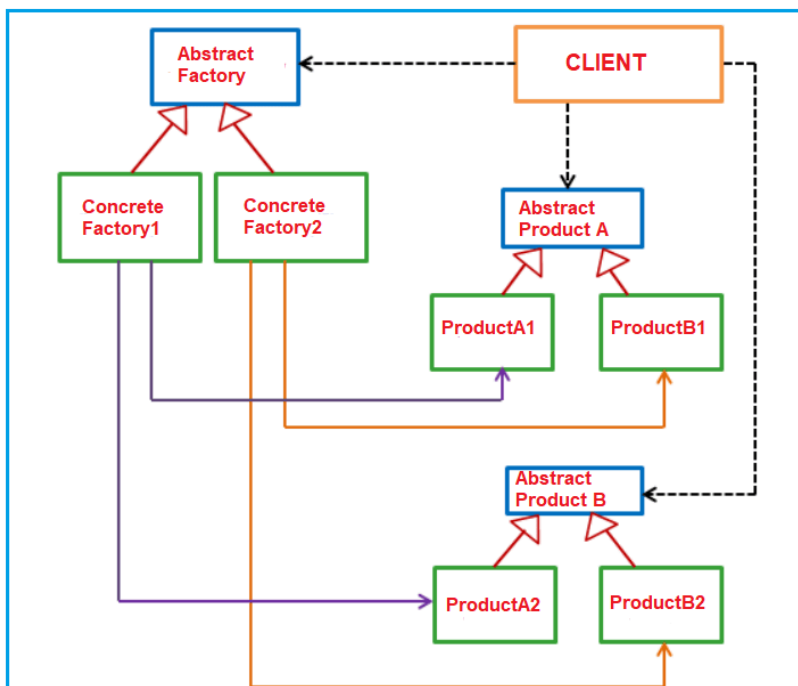
```

Fetching RegularBike Details..
Fetching RegularCar Details..
Fetching SportsBike Details..
Fetching SportsCar Details..

```

Abstract Factory Design Pattern UML (Class) Diagram:

Once we understand how to implement the Abstract Factory Design Pattern in C#, let us try to understand the UML (Unified Modeling Language) or Class diagram of the Abstract Factory Design Pattern. For a better understanding, please look at the following diagram, which shows the different components of the Abstract Factory Method Design Pattern. Here, on the right-hand side, I am comparing the Abstract Factory Design Pattern component with our example so that you can easily understand the different components of the Abstract Factory Design Pattern.



Abstract Factory Components in Our Example
 Client: Main Method of Program Class
 Abstract Product A: IBike.cs
 Abstract Product B: ICar.cs
 ProductA1: RegularBike.cs
 ProductB1: SportsBike.cs
 ProductA2: RegularCar.cs
 ProductB2: SportsCar.cs
 Abstract Factory: IVehicleFactory.cs
 Concrete Factory1: RegularVehicleFactory.cs
 Concrete Factory2: SportsVehicleFactory.cs

Let us understand each component of the Abstract Factory Design Pattern:

1. **Abstract Products:** These are going to be interfaces for creating abstract products. Here, we need to define the Operations a Product should have. In our example, it is IBike.cs and ICar.cs interfaces.
2. **Concrete Products:** These are the classes that implement the Abstract Product interface. In our example, RegularBike, SportsBike, RegularCar, and SportsCar classes are the Concrete Products.
3. **Abstract Factory:** This will be an interface for operations that create Abstract Product objects. In our example, it will be IVehicleFactory.

4. **Concrete Factory:** These classes implement the AbstractFactory interface and provide implementations for the interface methods. In our example, the concrete factory classes are RegularVehicleFactory and SportsVehicleFactory.
5. **Client:** This class will use the Abstract Product and Abstract Factory to create a family of products. In our example, it is the Main method of the Program class.

Abstract Factory Components in Our Example

1. **Client:** Main Method of Program Class
2. **Abstract Product A:** IBike.cs
3. **Abstract Product B:** ICar.cs
4. **ProductA1:** RegularBike.cs
5. **ProductB1:** SportsBike.cs
6. **ProductA2:** RegularCar.cs
7. **ProductB2:** SportsCar.cs
8. **Abstract Factory:** IVehicleFactory.cs
9. **Concrete Factory1:** RegularVehicleFactory.cs
10. **Concrete Factory2:** SportsVehicleFactory.cs

Abstract Factory Design Pattern Real-Time Example in C#:

Let us Implement the Abstract Factory Design Pattern Real-Time Example in C# step by step with one Real-Time Example. We want to implement one application for showing the Course Details. Here, we want to show two types of courses, i.e., Front End Courses and Back End Courses. Again, we must also show how students will learn these courses, i.e., Sources. In this case, the source can be Online or Offline. Let us proceed and see how we can implement this application using the Abstract Factory Design Pattern in C#.

Step 1: Creating Abstract Products

Here, we need to declare interfaces for creating Abstract Products. As we will create two types of familiar products, such as Courses and Sources, here we need to create two interfaces or abstract classes representing each abstract product type. Here, I am going to create two interfaces.

ICourse.cs

Create an interface named **ICourse.cs**, and copy and paste the following code. This is going to be one of the Abstract Products. Each distinct product of the Course product family should have a base interface. In this case, all variants of Course products (Front-End and Back-End) must implement this ICourse interface. As you can see, we have created the following interface with three abstract methods, which will be implemented by the subclasses (FrontEndCourse and BackEndCourse).

```
namespace AbstractFactoryRealTimeExample
{
    // The AbstractProductA interface
    // Each distinct product of the Course product family should have a
    base interface.
    // All variants of Course products must implement this ICourse
    interface.
    public interface ICourse
    {
        string GetCourseName();
        int GetCourseFee();
        string GetCourseDuration();
    }
}
```

ISource.cs

Create an interface named **ISource.cs** and copy and paste the following code. This is going to be our second Abstract Product. Each distinct product of the ISource product family should have a base interface. In this case, all variants of the Source products (Online and Offline) must implement this ISource interface. As you can see, we have created the following interface with one abstract method. That method will be implemented by the subclasses (Online and Offline) of the ISource interface.

```
namespace AbstractFactoryRealTimeExample
{
    // The AbstractProductB interface
    // Each distinct product of the Source product family should have a
    base interface.
    // All variants of Source products must implement this ISource
    interface.
    public interface ISource
    {
        string GetSourceName();
    }
}
```

Step 2: Creating Concrete Products

Now, we need to define the concrete product object or the actual product object that the corresponding concrete factory will create. You must remember that these concrete product classes must implement the Abstract Product Interface and provide implementation to the Interface methods. In our example, the concrete product class must implement the ISource or ICourse interface. So, as per our requirements, let us create four concrete products, i.e., FrontEndCourse, BackEndCourse, Online, and Offline.

FrontEndCourse.cs

Create a class file named **FrontEndCourse.cs** and copy and paste the following code. The following FrontEndCourse Product Belongs to the Course product family. As you can see, it implements the ICourse interface and provides implementations for all three ICourse interface methods.

```
namespace AbstractFactoryRealTimeExample
{
    // The ProductA1 class
    // Concrete Products are going to be created by corresponding Concrete
    Factories.
    // The following FrontEndCourse Product Belongs to the Course product
    family
    public class FrontEndCourse : ICourse
    {
        public string GetCourseName()
        {
            return "HTML, CSS, and Bootstrap";
        }
        public string GetCourseDuration()
        {
            return "6 Months";
        }
        public int GetCourseFee()
        {
            return 2000;
        }
    }
}
```

```
}  
}
```

BackEndCourse.cs

Create a class file named **BackEndCourse.cs** and copy and paste the following code. The following BackEndCourse Product Belongs to the Course product family. As you can see, it implements the ICourse interface and provides implementations for all three ICourse interface methods.

```
namespace AbstractFactoryRealTimeExample  
{  
    // The ProductB1 class  
    // Concrete Products are going to be created by corresponding Concrete  
    Factories.  
    // The following BackEndCourse Product Belongs to the Course product  
    family  
    public class BackEndCourse : ICourse  
    {  
        public string GetCourseDuration()  
        {  
            return "6 Months";  
        }  
        public int GetCourseFee()  
        {  
            return 1000;  
        }  
        public string GetCourseName()  
        {  
            return "C#, Java, and Python";  
        }  
    }  
}
```

Online.cs

Create a class file named **Online.cs** and copy and paste the following code. The following Online Product Belongs to the Source product family. As you can see, it implements the ISource interface and provides implementations for the GetSourceName method.

```
namespace AbstractFactoryRealTimeExample  
{  
    // The ProductA2 class  
    // Concrete Products are going to be created by corresponding Concrete  
    Factories.  
    // The following Online Product Belongs to the Source product family  
    public class Online : ISource  
    {  
        public string GetSourceName()  
        {  
            return "Dot Net Tutorials Zoom Classes";  
        }  
    }  
}
```

Offline.cs

Create a class file named **Offline.cs** and copy and paste the following code. The following OfflineProduct Belongs to the Source product family. As you can see, it implements the ISource interface and provides implementations for the GetSourceName method.

```
namespace AbstractFactoryRealTimeExample
{
    // The ProductB2 class
    // Concrete Products are going to be created by corresponding Concrete
    Factories.
    // The following Offline Product Belongs to the Source product family
    public class Offline : ISource
    {
        public string GetSourceName()
        {
            return "Dot Net Tutorials Class Room Training";
        }
    }
}
```

Step 3: Creating Abstract Factory

Now, we need to create an interface for operations that will create Abstract Product objects. In our example, it is going to be ISourceCourseFactory. So, create a class file named **ISourceCourseFactory.cs** and copy and paste the following. As you can see, this class contains two methods, i.e., GetSource and GetCourse. The GetSource method will create different variants of Sources, whereas the GetCourse method will create different variants of Courses. The point you need to remember is here that we are only declaring that the methods and return type of these methods will be abstract products, i.e., ISource and ICourse. Hence, from the child classes, we can create different objects (objects of subclasses of ISource and ICourse).

```
namespace AbstractFactoryRealTimeExample
{
    // The AbstractFactory interface
    // The Abstract Factory interface declares a set of methods that return
    different abstract products.
    // These products are called a family.
    // A family of products may have several variants
    public interface ISourceCourseFactory
    {
        //Abstract Product A
        ISource GetSource();
        //Abstract Product B
        ICourse GetCourse();
    }
}
```

Step 4: Creating Concrete Factories

We must create concrete factory classes that implement abstract factory interfaces to create concrete product objects. In our example, we will create two Concrete Classes, i.e. OnlineSourceCourseFactory and OfflineSourceCourseFactory.

OnlineSourceCourseFactory.cs

The OnlineSourceCourseFactory concrete class will create Online Source and Course Concrete Products. In our example, the Online Source and Course Concrete Products are Online and BackEndCourse. So, create a class file named **OnlineSourceCourseFactory.cs** and copy and paste the following code. If you notice, the signatures of the Concrete Factory's methods return an abstract product (ICourse) and (ISource), while inside the method, a concrete product (new Online and new BackEndCourse) is instantiated. So, the following Concrete Factory Produces Online Source and BackEndCourse Products belong to the same OnlineSourceCourse Family.

```
namespace AbstractFactoryRealTimeExample
{
    public class OnlineSourceCourseFactory : ISourceCourseFactory
    {
        public ISource GetSource()
        {
            return new Online();
        }
        public ICourse GetCourse()
        {
            return new BackEndCourse();
        }
    }
}
```

OfflineSourceCourseFactory.cs

The OfflineSourceCourseFactory concrete class will create Offline Source and Course Concrete Products. In our example, the Offline Source and Course Concrete Products are Offline and FrontEndCourse. So, create a class file named **OfflineSourceCourseFactory.cs** and then copy and paste the following code. If you notice, the signatures of the Concrete Factory's methods return an abstract product (ICourse) and (ISource), while inside the method, a concrete product (new Offline and new FrontEndCourse) is instantiated. So, the following Concrete Factory Produces Offline Source and FrontEndCourse Products belong to the same OfflineSourceCourse Family.

```
namespace AbstractFactoryRealTimeExample
{
    public class OfflineSourceCourseFactory : ISourceCourseFactory
    {
        public ISource GetSource()
        {
            return new Offline();
        }
        public ICourse GetCourse()
        {
            return new FrontEndCourse();
        }
    }
}
```

Note: In our example, ISourceCourseFactory provides an interface to the client for creating families of related or dependent objects. Here, we have two concrete implementations of that interface: the OnlineSourceCourseFactory and OfflineSourceCourseFactory classes. These two classes manufacture two different types of families of related objects: Courses and Sources. Online and BackEndCourse classes belong to the OnlineSourceCourse family. On the other hand, Offline and FrontEndCourse classes belong to the OfflineSourceCourse family.

Step 5: Client

The Client class uses AbstractFactory and AbstractProduct interfaces to create a family of related objects. In our example, it will be the Main method of the Program class. So, modify the Main method of the Program class as follows.

```

using System;
namespace AbstractFactoryRealTimeExample
{
    class Program
    {
        static void Main(string[] args)
        {
            // Fetch the Front End Course and Source Details
            // Creating OfflineSourceCourseFactory instance
            ISourceCourseFactory offlineSourceCourseFactory = new
OfflineSourceCourseFactory();
            //offlineSourceCourseFactory.GetCourse() will create and return
FronEndCourse object
            var course = offlineSourceCourseFactory.GetCourse();
            Console.WriteLine("Front End Course and Source Details");
            Console.WriteLine(course.GetCourseName());
            Console.WriteLine(course.GetCourseFee());
            Console.WriteLine(course.GetCourseDuration());
            //offlineSourceCourseFactory.GetSource() will create and return
Offline object
            var source = offlineSourceCourseFactory.GetSource();
            Console.WriteLine(source.GetSourceName());
            //Same steps for Online Course and Source Details
            Console.WriteLine("\n-----\n");
            Console.WriteLine("Back End Course and Source Details");
            ISourceCourseFactory onlineSourceCourseFactory = new
OnlineSourceCourseFactory();
            course = onlineSourceCourseFactory.GetCourse();
            Console.WriteLine(course.GetCourseName());
            Console.WriteLine(course.GetCourseFee());
            Console.WriteLine(course.GetCourseDuration());
            source = onlineSourceCourseFactory.GetSource();
            Console.WriteLine(source.GetSourceName());
            Console.ReadKey();
        }
    }
}

```

Output:

```

Front End Course and Source Details
HTML, CSS, and Bootstrap
2000
6 Months
Dot Net Tutorials Class Room Training
-----

Back End Course and Source Details
C#, Java, and Python
1000
6 Months
Dot Net Tutorials Zoom Classes

```

Abstract Factory Components in Our Example

1. **Client:** Main Method of Program Class
2. **Abstract Product A:** `ICourse.cs`
3. **Abstract Product B:** `ISource.cs`
4. **ProductA1:** `FrontEndCourse.cs`
5. **ProductB1:** `BackEndCourse.cs`
6. **ProductA2:** `Online.cs`
7. **ProductB2:** `Offline.cs`
8. **Abstract Factory:** `ISourceCourseFactory.cs`
9. **Concrete Factory1:** `OnlineSourceCourseFactory.cs`
10. **Concrete Factory2:** `OfflineSourceCourseFactory.cs`

Differences Between Abstract Factory and Factory Method Design Pattern in C#:

In C#, the Abstract Factory and Factory Method are creational design patterns that deal with object creation while promoting code decoupling and scalability. However, they serve different purposes and are used in distinct scenarios. The following are the key differences between these two patterns:

Level of Abstraction:

- Abstract Factory provides an interface for creating families of related or dependent objects without specifying their concrete classes.
- Factory Method is a method in a class that creates objects, and its subclasses decide which class to instantiate.

Purpose:

- Abstract Factory is used when your system needs to create multiple families of products, or you need a factory of factories.
- Factory Method is used when one product has multiple implementations, and the client does not need to know which it uses.

Complexity in Object Relationships:

- Abstract Factory deals with multiple products that are related and often designed to work together as a family.
- Factory Method focuses on one product at a time but provides a way to localize the decision-making about which concrete class to instantiate.

When to Use Abstract Factory:

- You need to create multiple related objects designed to be used together.
- You need to provide a library of products without exposing the implementation logic.
- You want to reveal just the interfaces of the products, not the implementations.

When to Use Factory Method:

- You need a method to create objects where the exact types aren't known until runtime.
- You want to give subclasses the responsibility to decide which class to instantiate.
- You want to localize the logic to instantiate a specific type of object within an overridden method.

In the next article, I will discuss the [Abstract Factory Design Pattern in Real-Time Examples in C#](#). In this article, I try to explain the **Abstract Factory Design Pattern** in C# with Examples. I hope this article will help you with your needs. I would like to have your feedback. Please post your feedback, questions, or comments about this article.